

Closing the Frontier–Small Model Gap with Domain-Specific Skills

Evidence from The Virtual Brain (TVB) benchmark suite

Marmaduke Woodman Autotvb Project

Aix-Marseille Université

2026

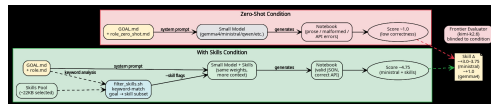
The Cost of Frontier Models

Frontier models (100B–1T parameters) excel at domain tasks:

- Write correct simulation code for Virtual Brain (TVB)
- Navigate complex APIs with hundreds of classes
- But: expensive, rate-limited, API-gated — unusable for many labs

Small models (8B–26B parameters) running locally:

- Cheap, fast, private — but lack domain knowledge
- Make systematic API errors (`sim.run()` returns a **list**, not generator)
- Cannot discover correct TVB patterns without guidance



The Capability Gap: Zero-Shot Small Models Struggle

Model	Params	Context	Zero-Shot Mean	With Skills	Skill Δ
rnj-1 (cloud)	8B	32K	3.75 / 5.0	3.94	+0.19
rnj-1 (local)	8B	32K	1.00	2.17	+1.17
qwen3.6	23B	128K	0.67	1.58	+0.92
ministral-14b-cloud	14B	128K	1.00	4.75	+3.75

Key Observation

Small models score **1.0/5.0** without guidance — they cannot even write valid notebook JSON. Skills raise scores to **4.75/5.0** (ministral), approaching frontier performance.

Hypothesis: Skills are Compressed Domain Expertise

Skills are not prompts. They are validated, composable artifacts distilled from measured failures.

What a skill captures:

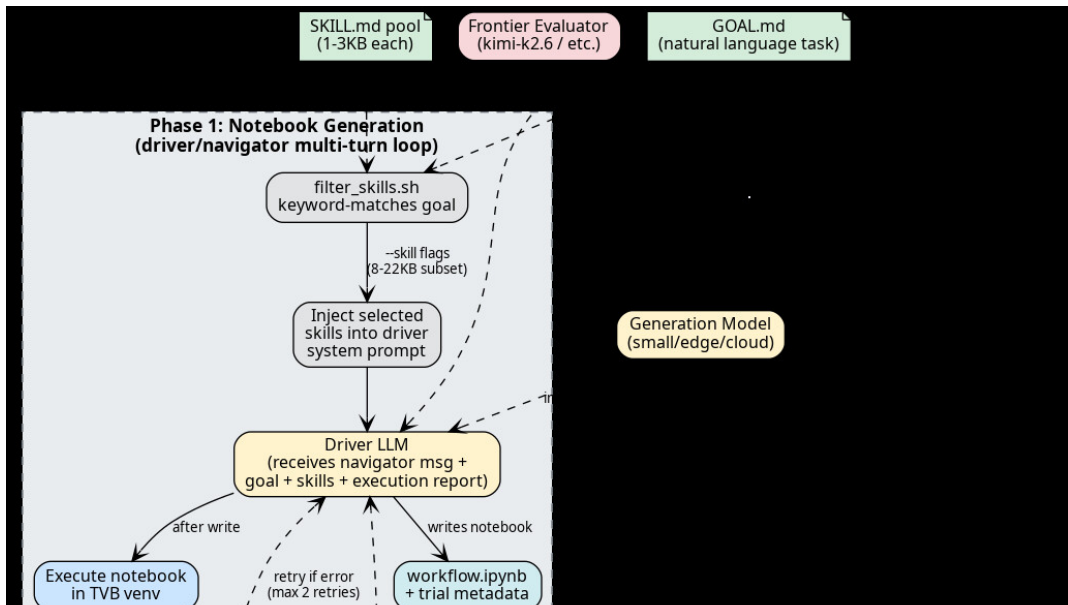
- boilerplate: Correct `sim.run()` return type (list of tuples)
- tvb-api-mappings: Paper param names $\rightarrow$ TVB trait names
- notebook-format: JSON cell structure, not prose markdown
- simulation-duration: BOLD ≥ 120 s for valid FC metrics

Each skill = **scar tissue from a measured failure pattern.**

How skills are created:

- 1 Frontier model generates notebook $\rightarrow$ scores it
- 2 Extract failure patterns (e.g., wrong API call)
- 3 Write targeted skill (1–3KB) addressing that pattern
- 4 Validate: small model + skill $\rightarrow$ new score
- 5 Iterate: skills continuously improve

Pipeline: Generation & Independent Evaluation



Example Skill: boilerplate (2.1KB)

This skill fixes the #1 failure pattern across all small models:

```
### TVB Simulation Boilerplate

**CRITICAL CORRECTNESS FACT**
simulator.run() returns a list of tuples, NOT a generator:

# CORRECT unpacking
(t1, d1), (t2, d2) = sim.run(...) # returns [(t1,d1), (t2,d2)]

# WRONG - crashes or returns garbage
t, data = sim.run(...) # tuple of arrays, not (time, data)
for t, d in sim.run(...): # TypeError: list is not iterable

**Correct connectivity setup**
conn.speed = numpy.array([v]) # BEFORE conn.configure()
conn.configure()
sim = simulator.Simulator(
    model=model, connectivity=conn,
    coupling=coupling, integrator=integrator,
    monitors=[temporalAverage], simulation_length=1000,
).configure()
```

Impact: rnj-1 correctness 3.9 → 4.4; ministril correctness 4.0 → 5.0.

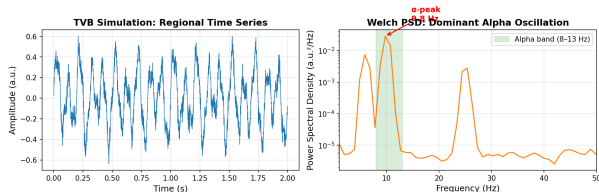
Example Goal, Notebook, and Skill Injection

Goal: *Analyze frequency content of a whole-brain simulation. Run a TVB simulation with a neural mass model, extract regional time series, compute power spectra via Welch's method, and identify dominant oscillatory frequencies.*

Skills injected (keyword-matched by `filter_skills.sh`):

- `driver/boilerplate` — correct `sim.run()` unpacking
- `driver/analysis` — Welch PSD, peak detection
- `driver/notebook-format` — valid JSON cell structure

Result (minstral-14b + skills):
correctness=5, code_quality=5,
scientific_validity=5, token_efficiency=4



Output from a successful notebook: alpha-band peak (10.0 Hz) dominates resting-state oscillations.

Abstract Goals: Skills Rescue Failed Zero-Shot Runs

rnj-1:8b local — **abstract goals** (no class-name hints, evaluated by kimi-k2.6)

Goal	Zero-Shot	With Skills	Skill Δ
analyze_power_spectra	1.00	3.00	+2.00
simulate_region_stimulus	failed	2.25	+2.25
stochastic_simulation	failed	2.25	+2.25
visual_erp	failed	2.00	+2.00
compare_connectivity_norm	1.75	2.25	+0.50
exploring_the_bold_monitor	2.25	2.50	+0.25

Per-Goal Mean (failures = 0)

Zero-shot: **0.67/5.0** With skills: **1.58/5.0** Δ : **+0.92** Success rate: 44% $\rightarrow$ 67%

Cloud Model Drilldown: ministral-14b-cloud is exceptional

Model	Cond.	scalar	cor.	c.q.	sci.	tok.
gemma4:31b-cloud	zero_shot	1.00	1	1	1	1
gemma4:31b-cloud	with_skills	2.00	1	2	2	3
ministral-14b-cloud	zero_shot	1.00	1	1	1	1
ministral-14b-cloud	with_skills	4.75	5	5	5	4

Both models fail zero-shot:

Write **prose markdown** instead of valid notebook JSON.

With skills: both produce valid JSON, but:

- gemma4:31b still makes API errors (wrong `sim.run()` unpacking)
- ministral-14b correct on all 4 dimensions

Key Insight

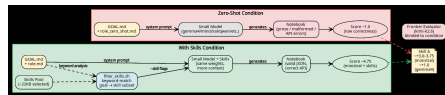
ministral-14b-cloud **outperforms** gemma4:31b-cloud despite **17B fewer parameters**.
Parameter count $\neq$ domain expertise.

Summary: Small Model + Skills + Prompts $\approx$ Frontier

Claim: Well-crafted domain skills can elevate small models (8B–26B) to frontier-level performance on specialized tasks.

Evidence:

- 1 **Skill Δ of +0.92–3.75** across models and goals
- 2 **Success rate doubles:** 44% $\rightarrow$ 67% (rnj-1), 0% $\rightarrow$ near-perfect (ministral)
- 3 **Skills fix systematic failures:** notebook format, API mappings, analysis patterns



Call to action:

- Transfer to other domains (Qiskit, FEniCS, Scanpy, ROS2)
- Standardize skill evaluation pipelines
- Make small models usable scientific tools